

Welcome to Week 3: Advice for Applying Machine Learning

You now have a toolbox of powerful algorithms (linear/logistic regression, neural networks). This week's goal is to learn how to **use these tools effectively** and efficiently.

The key to being an effective machine learning practitioner is making good decisions about **what to do next** to improve your model's performance.

The Core Problem: What to Try Next? 🤔

Imagine you've built a model (e.g., regularized linear regression to predict housing prices), but it makes **unacceptably large errors**.

You have many options. What should you try?

- Get more training examples.
 - Try a smaller set of features.
 - Try getting *more* (additional) features.
 - Try adding polynomial features (e.g., x_1^2 , $x_1 x_2$).
 - Try decreasing the regularization parameter, λ .
 - Try increasing the regularization parameter, λ .
-

The Challenge: Wasting Time ⌚

- Some of these options are extremely time-consuming. For example, a team might spend months collecting more training data.
 - This effort is wasted if collecting more data *wasn't* the right solution for the problem.
-

The Solution: Machine Learning Diagnostics 🛠️

Instead of guessing, you should run **diagnostics**.

- A **diagnostic** is a test you can run to gain insight into what is or isn't working with your learning algorithm.
- It provides guidance on which of the options (e.g., "get more data") is most likely to improve your performance.
- **Key Value:** Running a diagnostic (which may take some time to implement) can **save you weeks or even months** of wasted effort by helping you make better decisions.

Before we can use diagnostics, we must first have a clear way to evaluate our learning algorithm's performance.

Evaluating Your Model 🤖

When building a model, especially one with many features (making it hard to visualize), you need a systematic way to measure its performance. Simply fitting the data well isn't enough, as a model can **overfit** (fit the training data perfectly but fail to generalize to new data).

The Train-Test Split

The standard procedure is to split your dataset into two parts:

1. **Training Set (e.g., 70%–80%)**: The majority of the data. This portion is used to **train** the model (i.e., find the optimal parameters w and b).
2. **Test Set (e.g., 20%–30%)**: The remaining data. This portion is **held back** and used *only* to **evaluate** the trained model's performance on unseen data.

Notation

- **Training Set:**
 - m_{train} : Number of training examples.
 - Examples: $(x^{(1)}, y^{(1)}), \dots, (x^{(m_{\text{train}})}, y^{(m_{\text{train}})})$
 - **Test Set:**
 - m_{test} : Number of test examples.
 - Examples: $(x_{\text{test}}^{(1)}, y_{\text{test}}^{(1)}), \dots, (x_{\text{test}}^{(m_{\text{test}})}, y_{\text{test}}^{(m_{\text{test}})})$
-

Evaluation for Regression (e.g., Linear Regression)

1. **Train the Model**: Find parameters w and b by minimizing the **cost function**, $J(w, b)$, on the **training set**.
 - $J(w, b) = \frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} (f_{w,b}(x^{(i)}) - y^{(i)})^2 + \text{regularization term}$
 2. **Evaluate on Training Set (Training Error)**: Compute the average error on the **training set**, typically *without* the regularization term.
 - $J_{\text{train}}(w, b) = \frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} (f_{w,b}(x^{(i)}) - y^{(i)})^2$
 3. **Evaluate on Test Set (Test Error)**: Compute the average error on the **test set**, also *without* the regularization term. This is the primary measure of how well your model generalizes.
 - $J_{\text{test}}(w, b) = \frac{1}{2m_{\text{test}}} \sum_{i=1}^{m_{\text{test}}} (f_{w,b}(x_{\text{test}}^{(i)}) - y_{\text{test}}^{(i)})^2$
- An **overfit** model (like the 4th-order polynomial) will have a very low J_{train} but a very high J_{test} .
-

Evaluation for Classification (e.g., Logistic Regression)

For classification, you have two common ways to evaluate the model:

Method 1: Using the Loss Function

This is the same as regression, but using the logistic loss (cross-entropy) instead of squared error.

1. **Train the Model:** Minimize the regularized **cost function** $J(w, b)$ on the training set.
2. **Evaluate (Test Error):** Compute the average **test error**, J_{test} , using the logistic loss on the test set.

$$\circ J_{test}(w, b) = -\frac{1}{m_{test}} \sum_{i=1}^{m_{test}} \left[y_{test}^{(i)} \log(f_{w,b}(x_{test}^{(i)})) + (1 - y_{test}^{(i)}) \log(1 - f_{w,b}(x_{test}^{(i)})) \right]$$

3. **Evaluate (Training Error):** Compute the average **training error**, J_{train} , using the logistic loss on the training set.

Method 2: Misclassification Error (More Common)

A more intuitive metric for classification is the **misclassification error**, which is the *fraction* of examples the model got wrong.

- **Test Error (J_{test}):** The fraction of test set examples where the model's prediction $\hat{y}$ does not equal the actual label y_{test} .
 - *Example:* If the model misclassifies 3 out of 30 test examples, $J_{test} = \frac{3}{30} = 0.1$ (or 10%).
- **Training Error (J_{train}):** The fraction of training set examples that the model misclassified.

Splitting your data into training and test sets is the first step in systematically evaluating your model and, as you'll see next, automatically selecting the best model configuration.

The Problem: How to Choose a Model?

The test set (J_{test}) gives us a good estimate of how well a *single* model generalizes to new data. But what if we are trying to decide *which* model to use in the first place?

For example, we might be choosing between 10 different models:

- Model 1: 1st-degree polynomial ($d = 1$)
 - Model 2: 2nd-degree polynomial ($d = 2$)
 - ...
 - Model 10: 10th-degree polynomial ($d = 10$)
-

A Flawed Procedure

A tempting, but flawed, procedure would be:

1. Train all 10 models on the **training set**.
2. Evaluate all 10 models on the **test set**.
3. See which model had the lowest test error (e.g., $d = 5$).
4. Report this lowest test error as our generalization error.

Why this is flawed: We have now used the test set to make a decision—we "fit" the parameter d (the degree of the polynomial) to our test set. By picking the *best* result from this set, our reported test error (J_{test}) is now an **overly optimistic** (artificially low) estimate of how the model will do on new, unseen data.

The Solution: The 3-Way Data Split

To correctly perform model selection, we split our data into **three** separate sets.

1. **Training Set (e.g., 60% of data)**
 - **Purpose:** To train the parameters (w, b) for *each* of our candidate models.
 2. **Cross-Validation (CV) Set (e.g., 20% of data)**
 - **Purpose:** To select the best model. We use this set to "cross-check" the performance of the different models.
 - Also known as the **validation set** or **development (dev) set**.
 3. **Test Set (e.g., 20% of data)**
 - **Purpose:** To provide a final, **unbiased estimate of the generalization error** of our *one* chosen model. This set is "untouched" until the very end.
-

The Correct Model Selection Procedure

Here is the standard, correct procedure for choosing the best model and evaluating it fairly:

1. **Train all candidate models** (e.g., all 10 polynomial models) on the **training set**.
2. **Evaluate all trained models** on the **cross-validation (CV) set**.
3. **Pick the one model** that performed the best (had the lowest error) on the **CV set** (J_{cv}). For example, let's say the $d = 4$ polynomial had the lowest CV error.
4. **Finally, take your single chosen model** (the $d = 4$ model) and evaluate it *one time* on the **test set**.
5. Report the resulting J_{test} as the unbiased estimate of your model's generalization error.

This same procedure is used for choosing any model parameters, such as the architecture of a neural network (e.g., comparing a small, medium, and large network to see which performs best on the CV set).

Best Practice: All decisions—fitting parameters (w, b), choosing the model (polynomial degree d , network architecture, regularization λ)—should be made using *only* the **training set** and **cross-validation set**. The **test set** should only be used once at the very end to get a final performance metric.